Chapter 3. Google Gemini 연동하기

> ⏱️ **배정 시간 50분** — API Key 확인 5분 · 노드 구조 10분 · 프롬프트 작성법 10분 · 실습 3-1 15분 · 실습 3-2 10분

📋 학습 목표

이 챕터를 마치면 다음을 할 수 있습니다.

- [ ] 1일차에 발급한 Gemini API Key를 n8n Credential로 등록하고 워크플로우에서 사용할 수 있다
- [ ] Basic LLM Chain과 Chat Model 노드의 연결 구조를 이해할 수 있다
- [ ] Temperature가 결과에 미치는 영향을 직접 확인하고 설명할 수 있다
- [ ] 태그를 활용해 입력과 지시를 구분하고 출력 형식을 지정할 수 있다
- [ ] 회의록을 요약하고 액션 아이템을 추출한 뒤 공지 메일 초안까지 만드는 워크플로우를 완성할 수 있다

📖 이 장의 용어

용어	영문 · 약어 풀이	설명
API Key	API 키, 인증 키	Google이 "누가 호출했는지"를 확인하는 비밀 문자열. `AIza...`로 시작
Credential	자격 증명	API Key를 n8n에 저장해 두는 것. 워크플로우 안에 키를 직접 적지 않기 위해 사용
Basic LLM Chain	—	프롬프트 하나를 넣으면 텍스트 하나가 나오는 가장 단순한 AI 노드. 도구 호출과 대화 기억 없음
Chat Model	채팅 모델 (서브 노드)	실제 LLM을 호출하는 노드. Basic LLM Chain이나 AI Agent의 아래쪽 커넥터에 붙여 사용
Flash / Pro	—	Gemini 모델 등급. Flash는 빠르고 저렴, Pro는 고성능이나 무료 한도가 적거나 없음
토큰	Token	LLM이 텍스트를 처리하는 최소 단위. 사용량과 응답 길이 제한(Maximum Number of Tokens)의 기준
Temperature	온도	응답의 무작위성을 정하는 값(0~1). 낮을수록 매번 비슷한 답, 높을수록 다양한 답

용어	영문 · 약어 풀이	설명
Top K / Top P	—	다음 단어 후보를 몇 개(K) 또는 확률 합 얼마까지(P)로 제한할지. 이 과정에서는 기본값 유지
Safety Settings	안전 설정	유해 콘텐츠 필터의 강도
프롬프트	Prompt	LLM에 전달하는 입력. 역할·작업 지시·제약·출력 형식 네 요소로 구성
태그	Tag	`<minutes>...</minutes>`처럼 텍스트의 범위를 표시하는 기호. 입력 데이터와 지시를 구분하고 출력 형식을 정할 때 사용
액션 아이템	Action Item	회의에서 정해진 할 일. 담당자·내용·기한으로 구성
Code 노드	—	JavaScript 코드를 직접 실행하는 노드. 다른 노드로 표현하기 어려운 처리에 사용
정규식	Regular Expression, RegExp	문자 패턴을 찾는 표기법. 이 장의 코드에서는 "두 태그 사이의 모든 글자"를 찾는 데 사용
후처리	Post-processing	LLM 응답을 받은 뒤 필요한 부분만 뽑거나 형식을 바꾸는 작업

3.1 Gemini API Key 등록

3.1.1 API Key 확인

> 🔗 **1일차 연결**: 1일차 Chapter 2에서 Google AI Studio에서 API Key를 발급해 `my-adk/.env`에 저장했습니다. 오늘은 **같은 키를 n8n에 등록**합니다. 키를 메모해 두지 않았다면 aistudio.google.com/apikey에서 목록의 키 이름을 클릭해 다시 확인하거나, 새로 발급받습니다.

새로 발급해야 하는 경우

1. aistudio.google.com/apikey 접속 후 Google 계정으로 로그인
2. **Create API Key** 클릭
3. 프로젝트를 선택하거나 새 프로젝트에 생성
4. 생성된 키를 복사해 메모장에 저장

키는 `AIza...`로 시작하는 긴 문자열입니다.

무료 사용 한도

이 과정은 **Flash 계열 모델**을 사용합니다. 무료 사용량은 Google이 수시로 조정하며 모델마다 다르므로 교재에 수치를 적지 않습니다. 당일 기준 한도는 강사가 수업 시작 시 안내하

며, 본인 계정의 한도는 Google AI Studio에서 확인할 수 있습니다.

> ⚠️ **실습 시 주의**
> - 무료 티어에서는 입력한 프롬프트와 응답이 Google의 모델 개선에 활용될 수 있습니다. **실제 회의록이나 회사 기밀은 실습에 넣지 마세요.** 이 교재의 예시 텍스트는 모두 가상의 내용입니다.
> - API Key는 워크플로우 안에 직접 입력하지 말고, 반드시 다음 단계의 Credential 기능으로만 등록합니다.
> - Agent 실습(Chapter 4~6)에서는 질문 하나에 요청이 2~3회 발생합니다. 한도가 적은 날에는 테스트 횟수를 조절하세요.

3.1.2 n8n에 Credential 등록

1. n8n 좌측 사이드바 → **Credentials** → **Add Credential**
2. 검색창에 `Gemini` 입력 → **Google Gemini(PaLM) API** 선택
3. `API Key` 필드에 복사한 키를 붙여넣기
4. `Host`는 기본값 `https://generativelanguage.googleapis.com` 그대로 둡니다
5. Credential 이름을 알아보기 쉽게 지정 (예: `Gemini - 수업용`) 후 **Save**

저장하면 화면에 연결 성공 표시가 나타납니다. 한 번 등록해두면 이후 모든 Gemini 관련 노드에서 드롭다운으로 선택만 하면 됩니다.

확인 체크리스트

- [ ] Gemini API Key 확보 (1일차 키 또는 신규 발급)
- [ ] n8n Credentials에 Google Gemini(PaLM) API 등록 완료

> ⚠️ `API key not valid` 오류가 나면 키를 복사할 때 앞뒤 공백이 섞였을 가능성이 큽니다. 메모장에 붙여넣어 확인한 뒤 다시 입력해보세요.

3.2 Basic LLM Chain 노드

3.2.1 노드 구조 — 노드 두 개가 한 세트

Basic LLM Chain은 "프롬프트를 넣으면 텍스트가 나오는" 가장 단순한 AI 노드입니다. Chapter 4에서 다룰 AI Agent와 달리 도구 호출도, 대화 기억도 없이 **한 번 입력 → 한 번 출력**으로 끝납니다. 1일차로 치면 Tool이 없는 Agent(Chapter 2)에 해당합니다.

여기서 처음 만나는 낯선 구조가 하나 있습니다. **Basic LLM Chain 노드는 혼자서는 동작하지 않습니다.** 노드 아래쪽에 달린 커넥터에 실제로 생각을 담당할 **Chat Model 노드를 따로**

연결해야 합니다.

```
Basic LLM Chain          ← "무엇을 시킬지"를 담당
      |
      └── (하단 Model 커넥터) ── Google Gemini Chat Model   ← "실제로 생각"을 담당
```

지금까지 쓴 노드들은 왼쪽에서 오른쪽으로만 연결했는데, AI 관련 노드는 **아래쪽으로도 연결**합니다. 이 구조는 Chapter 4의 AI Agent에서도 똑같이 반복되니 여기서 익혀두면 편합니다.

설정 절차

1. 워크플로우에 `Basic LLM Chain` 노드 추가
2. 노드 **하단**의 `Model` 커넥터에 있는 `+` 클릭
3. `Google Gemini Chat Model` 검색 후 추가
4. 3.1.2에서 등록한 Credential 선택
5. `Model` 드롭다운에서 **Flash 계열 모델** 선택 (1일차에 사용한 것과 같은 모델이면 됩니다)

3.2.2 모델 파라미터

Google Gemini Chat Model 노드의 `Add Option`에서 조정할 수 있습니다.

파라미터	역할	실습 권장값
Model	사용할 모델	Flash 계열
Maximum Number of Tokens	응답 최대 길이	`1024`
Sampling Temperature	응답의 일관성/다양성	요약·추출 `0.2`, 메일 초안 `0.5`
Top K / Top P	다음 단어 후보 범위	기본값 유지
Safety Settings	유해 콘텐츠 필터 강도	기본값 유지

Temperature 감 잡기

값	결과 경향	적합한 작업
0.0 ~ 0.3	매번 비슷한 답변, 정확성 중시	요약, 분류, 데이터 추출
0.4 ~ 0.7	적당한 다양성	톤 변환, 이메일 초안
0.8 ~ 1.0	매번 다른 창의적 답변	아이디어 발상, 카피라이팅

> 💡 Temperature는 **Chat Model 노드**의 설정입니다. 작업마다 다른 값을 쓰려면 Chat Model 노드를 작업 수만큼 따로 두어야 합니다. 이 챕터의 실습이 그렇게 구성되어 있습

> 니다. 설정이 같다면 하나의 Chat Model을 여러 Chain이 공유할 수도 있습니다.

✅ 체크포인트 질문

- Basic LLM Chain 노드만 추가하고 바로 실행할 수 있을까요? → **아니오, 하단 Model 커넥터에 Chat Model을 연결해야 함**
- 같은 입력에 항상 일관된 요약을 얻으려면 Temperature는? → **낮게(0.2~0.3)**
- 요약은 0.2, 메일 초안은 0.5로 하려면? → **Chat Model 노드를 두 개 만들어 각각 연결**

3.3 프롬프트 작성법

> 🔗 **1일차 연결**: 1일차 Chapter 6에서 Role · Output Format · Constraint로 Instruction을 설계했습니다. 아래 4요소는 같은 내용을 n8n 프롬프트 관점에서 정리한 것입니다. "작업 지시"만 추가되었습니다.

3.3.1 좋은 프롬프트의 네 가지 구성

프롬프트를 막연히 쓰면 결과도 막연해집니다. 아래 네 가지를 순서대로 채우면 대부분 잘 동작합니다.

```
1. 역할 정의 — 너는 무엇을 하는 AI인가            (1일차: Role)
2. 작업 지시 — 구체적으로 무엇을 해야 하는가
3. 제약 조건 — 하지 말아야 할 것                  (1일차: Constraint)
4. 출력 형식 — 결과를 어떤 모양으로 줄 것인가     (1일차: Output Format)
```

예시 (회의록 요약용)

```
당신은 회의록을 정리하는 사내 비서입니다.
입력된 회의록을 3줄 이내로 핵심만 요약하세요.
회의록에 없는 결정 사항을 추가하지 마세요.
요약은 <summary> 태그 안에 작성하세요.
```

3.3.2 태그로 입력과 지시 구분하기

프롬프트 안에 "지시사항"과 "처리할 데이터"가 섞여 있으면 모델이 헷갈립니다. 특히 데이터 안에 명령처럼 보이는 문장이 들어 있으면 그것까지 지시로 착각할 수 있습니다. 회의록에는 "~하기로 함", "~해 주세요" 같은 문장이 많아 특히 그렇습니다.

이걸 막는 가장 간단한 방법이 **태그로 데이터를 감싸는 것**입니다.

```
다음 <minutes> 태그 안의 회의록을 요약하세요.

<minutes>
{{ $json.minutes_text }}
</minutes>

요약 결과는 <summary> 태그 안에 작성하세요.
```

출력 형식도 태그로 지정하면 좋습니다

```
아래 형식을 반드시 지켜서 답변하세요.

<summary>
(3줄 이내 요약)
</summary>
<action_items>
(담당자 | 할 일 | 기한 — 한 줄에 하나씩)
</action_items>
```

이렇게 받아두면 다음 노드에서 `<summary>` 안의 값만 뽑아내는 후처리가 쉬워집니다. Chapter 7의 이메일 자동 분류에서 이 방식을 그대로 사용합니다.

3.3.3 역할 부여

"너는 ○○이다"라는 한 문장만으로도 답변의 톤과 깊이가 달라집니다.

역할 부여	결과 경향
"너는 회의록을 정리하는 사내 비서다"	사실 위주, 간결
"너는 팀 공지를 작성하는 담당자다"	정중하고 격식 있는 표현
"너는 신입사원에게 설명하는 선배다"	쉬운 단어, 배경 설명 추가

✅ 체크포인트 질문

- 입력 데이터와 지시사항이 섞여 모델이 혼동하는 것을 막으려면? → **태그로 입력 데이터를 감싸기**
- "너는 ○○이다"를 프롬프트에 넣는 이유는? → **답변의 관점과 톤을 특정 역할에 맞게 유도하기 위해**

3.4 실습 — 회의록 정리 도구

> 🎯 **목표**: 회의록 텍스트를 입력받아 ① 요약과 액션 아이템을 추출하고 ② 그 결과로 팀 공지 메일 초안을 만든 뒤 ③ 태그를 제거해 바로 쓸 수 있는 형태로 정리한다.
> ⏱️ **소요 시간 25분** — 실습 3-1(요약·액션 아이템) 15분 → 실습 3-2(메일 초안·정리) 10분

이 실습이 해결하는 업무

회의가 끝나면 누군가 회의록을 정리해 요약과 할 일을 뽑고, 참석하지 못한 사람에게 공지를 보냅니다. 이 세 단계를 한 번에 처리합니다.

완성 흐름

```
Manual Trigger → 회의록 입력(Edit Fields) → 회의록 요약(Basic LLM Chain) → 메일 초안 작성(Basic LLM Chain) → 결과 정리(Code)
```

3.4.1 실습 3-1. 회의록 요약과 액션 아이템 추출

원문 준비

1. 새 워크플로우 생성, 이름을 `실습3_회의록정리`로 설정
2. `Manual Trigger` 노드 추가
3. `Edit Fields (Set)` 노드를 추가하고 이름을 `회의록 입력`으로 변경, 필드 생성

Field Name	Type	Value
`minutes_text`	String	아래 예시 텍스트 붙여넣기 (Expression 모드 아님, 일반 텍스트)

```
[주간 운영회의] 10월 23일(목) 14:00~14:40, 참석: 김과장, 박대리, 이주임, 최사원

1. 신규 입사자 온보딩
- 11월 3일 입사 예정 2명. 노트북과 계정은 10월 30일까지 준비(박대리).
- 온보딩 가이드 문서가 작년 버전이라 수정 필요. 이주임이 10월 29일까지 초안 작성.

2. 4분기 사무용품 구매
- 복합기 토너 재고 부족. 최사원이 금주 내 발주.
- 해외 직구 품목(모니터암 10개)은 환율 1,400원 넘으면 보류하기로 함. 김과장이 매주 월요일 환율 확인.

3. 기타
- 11월 전사 워크숍 장소 후보 3곳 비교표를 다음 회의까지 준비(박대리).
- 다음 회의: 10월 30일(목) 14:00
```

요약 단계

4. `회의록 입력` 뒤에 `Basic LLM Chain` 노드 추가, 이름을 `회의록 요약`으로 변경

5. 하단 Model 커넥터에 `Google Gemini Chat Model` 연결 → Credential과 Flash 계열 모델 선택
6. Chat Model 노드의 `Add Option`에서 Sampling Temperature를 `0.2`, Maximum Number of Tokens를 `1024`로 설정
7. `회의록 요약` 노드 상단의 `Source for Prompt (User Message)` 드롭다운을 `Define below`로 변경 (아래 주의사항 반드시 확인)
8. 나타난 `Prompt (User Message)` 필드에 아래 프롬프트 입력
9. ▶ 버튼으로 실행 → OUTPUT에 `<summary>`, `<action_items>` 태그가 포함된 응답이 나오는지 확인

> ⚠️ **7번 단계를 빠뜨리면 반드시 오류가 납니다.**
> 기본값인 `Connected Chat Trigger Node`는 이전 노드에 `chatInput`이라는 필드가 있다고 가정합니다. 우리는 `minutes_text`라는 다른 이름을 썼으므로 그대로 두면 `No prompt specified` 오류가 뜹니다. **Basic LLM Chain을 추가할 때마다 매번 바꿔야 합니다.**

8번 단계에 입력할 프롬프트

```
당신은 회의록을 정리하는 사내 비서입니다.
다음 <minutes> 태그 안의 회의록을 처리하세요.

<minutes>
{{ $json.minutes_text }}
</minutes>

1. 회의에서 결정된 내용을 3줄 이내로 요약하세요.
2. 할 일(액션 아이템)을 모두 추출하세요. 담당자와 기한이 회의록에 적혀 있으면 그대로 쓰고, 없으면 "미정"이라고 쓰세요.
회의록에 없는 내용을 추가하지 마세요.

결과는 아래 형식을 반드시 지켜서 출력하세요.
<summary>
(3줄 이내 요약)
</summary>
<action_items>
담당자 | 할 일 | 기한
담당자 | 할 일 | 기한
</action_items>
```

🔬 Temperature를 직접 체감해보기 (3분)

1. 현재 설정(`0.2`)에서 ▶ 버튼을 **연속으로 3번** 눌러 결과를 비교합니다 → 거의 같은 문장이 나옵니다
2. Chat Model 노드에서 Sampling Temperature를 `0.9`로 바꿉니다

3. 다시 ▶ 버튼을 **연속으로 3번** 눌러 비교합니다 → 매번 다른 표현으로 요약되고, 액션 아이템 개수가 달라지기도 합니다
4. 확인 후 Sampling Temperature를 `0.2`로 되돌립니다

요약이나 추출처럼 결과가 일정해야 하는 작업에서 Temperature를 높이면 안 되는 이유가 이것입니다.

3.4.2 실습 3-2. 공지 메일 초안 작성과 결과 정리

메일 초안 단계

1. `회의록 요약` 뒤에 `Basic LLM Chain` 노드를 하나 더 추가, 이름을 `메일 초안 작성`으로 변경
2. 하단 Model 커넥터에 **새로운** `Google Gemini Chat Model` 노드를 추가해 연결 (기존 노드를 재사용하지 않습니다)
3. 새 Chat Model 노드에서 Credential·Flash 모델 선택, Sampling Temperature를 `0.5`로 설정
4. `Source for Prompt (User Message)`를 **다시 한 번** `Define below`로 변경 (새로 추가한 노드는 기본값으로 돌아가 있습니다)
5. `Prompt (User Message)`에 아래 프롬프트 입력
6. ▶ 버튼으로 실행 → `<email_draft>` 태그 안에 공지 메일 형태의 문장이 나오는지 확인

```
당신은 팀 공지 메일을 작성하는 담당자입니다.
다음 <result> 태그 안에는 회의 요약과 액션 아이템이 들어 있습니다.

<result>
{{ $json.text }}
</result>

이 내용을 바탕으로 회의에 참석하지 못한 팀원에게 보낼 공지 메일 본문을 작성하세요.
- 정중한 비즈니스 문체를 사용하세요.
- 첫 줄에 회의명과 일시를 적고, 결정 사항과 할 일을 구분해서 정리하세요.
- 할 일에는 담당자와 기한을 반드시 포함하세요.
- <result>에 없는 내용은 추가하지 마세요.

결과는 <email_draft> 태그 안에만 작성하세요.
```

> 💡 여기서 `$json.minutes_text`가 아니라 `$json.text`인 이유는, 바로 앞 노드가 Edit Fields가 아니라 **Basic LLM Chain**이기 때문입니다. Basic LLM Chain의 출력 필드명은 기본적으로 `text`입니다. 헷갈리면 앞 노드의 OUTPUT 패널을 열어 실제 필드명을 확인하세요.

결과 정리 (Code 노드)

마지막으로 태그를 제거하고 세 결과(요약, 액션 아이템, 메일 초안)를 각각의 필드로 정리합니다. Code 노드는 JavaScript 코드를 직접 실행하는 노드이며, 태그 사이의 텍스트를 찾는 데는 정규식(Regular Expression, 문자 패턴을 찾는 표기법)을 사용합니다. 요약과 액션 아이템은 `회의록 요약` 노드의 출력에, 메일 초안은 바로 앞 노드의 출력에 들어 있으므로 두 노드를 모두 참조합니다.

7. `메일 초안 작성` 뒤에 `Code` 노드 추가, 이름을 `결과 정리`로 변경
8. 아래 코드 입력

```
// 1) 두 노드의 출력을 가져옵니다
const summaryRaw = $('회의록 요약').item.json.text;   // 요약 + 액션 아이템
const emailRaw = $json.text;                          // 메일 초안 (바로 앞 노드)

// 2) 태그 사이의 내용만 뽑아내는 함수
const pick = (text, tag) => {
  const match = text.match(new RegExp('<' + tag + '>([\\s\\S]*?)</' + tag + '>'));
  return match ? match[1].trim() : '';
};

// 3) 세 필드로 정리해서 반환
return [{
  json: {
    summary: pick(summaryRaw, 'summary'),
    action_items: pick(summaryRaw, 'action_items'),
    email_draft: pick(emailRaw, 'email_draft') || emailRaw
  }
}];
```

줄	역할
`$('회의록 요약').item.json.text`	Chapter 1에서 배운 "다른 노드의 값 가져오기"를 코드 안에서 쓴 것
`pick` 함수	`<태그>`와 `</태그>` 사이의 텍스트를 찾아 반환. 태그가 없으면 빈 문자열
`|| emailRaw`	모델이 `<email_draft>` 태그를 빠뜨린 경우 응답 원문을 그대로 사용

> ⚠️ `$('회의록 요약')`의 노드 이름은 **캔버스의 실제 노드 이름과 정확히 일치**해야 합니다. 이름을 다르게 지었다면 코드도 함께 고칩니다.

9. ▶ 버튼으로 실행 → `summary`, `action_items`, `email_draft` 세 필드에 태그 없는 깔끔한 내용이 나오는지 확인

🎉 완성 확인

OUTPUT에 아래와 비슷한 세 필드가 나오면 실습 완료입니다. (문장은 실행마다 조금씩 다릅니다)

```
{
  "summary": "신규 입사자 2명의 장비·계정 준비와 온보딩 가이드 수정을 진행한다. ...",
  "action_items": "박대리 | 노트북·계정 준비 | 10월 30일\n이주임 | 온보딩 가이드 초안 | 10월 29일\n...",
  "email_draft": "안녕하세요, 팀원 여러분.\n10월 23일(목) 주간 운영 회의 내용을 공유드립니다. ..."
}
```

마지막으로 **Save**합니다. `action_items`는 "담당자 | 할 일 | 기한" 형식이므로, Chapter 6에서 배울 Data Table이나 사내 업무 도구에 그대로 넣을 수 있는 구조입니다.

응용 아이디어 (수업 후)

- `회의록 입력` 노드 대신 Chapter 2의 Webhook Trigger를 쓰면, 사내 폼에 회의록을 붙여넣는 즉시 처리됩니다.
- `결과 정리` 뒤에 Chapter 2의 Send Email 노드를 붙이면 공지 메일이 실제로 발송됩니다. 단, AI가 쓴 초안을 사람이 확인하는 단계 없이 자동 발송하는 것은 권장하지 않습니다.

📌 Chapter 3 요약

개념	핵심 내용
API Key	1일차 키 재사용. Credentials → Add Credential → Google Gemini(PaLM) API
Basic LLM Chain 구조	노드 단독으로는 동작 불가. **하단 Model 커넥터에 Chat Model 연결 필수**
Source for Prompt	노드를 추가할 때마다 매번 `Define below`로 변경해야 함

개념	핵심 내용
모델 선택	Flash 계열. 1일차와 같은 모델
Temperature	Chat Model 노드의 설정. 작업마다 다르게 쓰려면 노드를 따로 둠
프롬프트 4요소	역할 + 작업 지시 + 제약 조건 + 출력 형식 (1일차 Role·Constraint·Format)
태그 활용	입력을 감싸 지시와 분리, 출력 형식을 지정해 후처리를 쉽게 만듦
출력 필드 `text`	Basic LLM Chain의 결과는 `$json.text`에 담김
실습 3-1·3-2	회의록 → 요약·액션 아이템 → 공지 메일 초안 → Code 노드로 세 필드 정리

다음 Chapter 예고: 지금까지의 AI는 시키는 순서대로만 움직였습니다. Chapter 4에서는 **대화를 기억하고, 필요한 도구를 스스로 골라 쓰는** AI Agent 노드로 견적과 일정을 계산하는 사내 업무 비서를 만듭니다.